

Overview

A pnpm monorepo providing a centralised admin portal for universities to manage course data. A React/Vite frontend talks to a Python FastAPI backend (port 8080). Background scraping is handled by Celery workers (Redis broker) with Gemini AI fallback. All data lives in PostgreSQL.

LAYER 1 -- Frontend

Browser / Admin Portal (artifacts/university-portal/)

Framework: React 18 + Vite | Language: TypeScript 5.9
UI: shadcn/ui (Radix primitives) + Tailwind CSS
Data fetching: TanStack React Query (caching, mutations, refetch)
Routing: Wouter (lightweight client-side SPA routing)
Auth: HTTPOnly JWT cookie ('session', 7-day) | AuthGuard on all protected routes
Auth context: src/context/auth.tsx -- useAuth() hook + AuthProvider
Pages: Dashboard, Universities, Courses, Scraper Jobs, Import History, Scraped Changes
API comms: Relative URLs via shared Nginx proxy | Orval-generated React Query hooks

LAYER 2 -- API Server

FastAPI :8080 (Python / Uvicorn)

Language: Python 3.12 | ASGI: Uvicorn
Dev: port 8080 (--reload flag)
Prod: port 8000, behind Nginx
Nginx: /api -> 127.0.0.1:8000
Entry: backend-py/app/main.py
DB: asyncpg + SQLAlchemy async
CRITICAL: DB URL must use 127.0.0.1
NOT localhost (SSL/DNS bug)

Auth (JWT Cookie)

POST /api/auth/login -- set cookie
POST /api/auth/logout -- clear cookie
GET /api/auth/me -- session check
Cookie: 'session' (HTTPOnly, 7-day)
Default: admin@university-portal.local
Override via ADMIN_EMAIL / ADMIN_PASSWORD
env vars on the server

API Routers (backend-py/app/routers/)

courses.py -- CRUD + bulk CSV export (NUMERIC -> float fix for asyncpg)
universities.py -- CRUD + course listing per university
scraper_jobs.py -- trigger scrape, poll status, approve/reject scraped changes
bulk_import.py -- Excel (.xlsx) upload -> validation -> import_jobs table
dashboard.py -- stats, courses by degree level, upcoming intakes, recent changes
change_history.py -- field-level diff log (before/after values) for audit trail
auth.py -- login / logout / me endpoints

LAYER 3 -- Background Workers

Celery Worker (backend-py/app/tasks/)

Concurrency: 4 workers | Queue: -Q scrape
Broker/backend: Redis :6379 (in-memory)
Tasks:
run_scrape(university_id, config)
run_repair(university_id)
bulk_approve(university_id)
Start: celery -A app.tasks.celery_app worker
--concurrency=4 --beat --loglevel=info

Redis :6379 + Celery Beat

Redis: in-memory task broker only
Bind: 127.0.0.1:6379
No persistence (--save "")
Beat schedule:
nightly_sweep @ 02:00 UTC
Step 1: capture_baseline.py snapshot
Step 2: regression_sweep.py diff
Step 3: drift alert if unexpected diffs

LAYER 4 -- Scraper Engine

LAYER 5 -- Field Extractors (backend-py/app/services/scrapper/extractors/)

course_name.py

DOM title extraction
Strip title suffixes
Institution tail remove
YAML: strip_title_suffixes

duration.py

DOM structural pass
Sentence tournament
YAML: reject_sentence
_patterns (new)
Filters 'up to N years'

fee.py

International tuition
PDF table matching
CRICOS-matched rows
Per-course PDF parse

english_test.py

IELTS/PTE/TOEFL/CAE/DET
Band score extraction
PTE host blacklist
Overall + sub-band scores

location.py

Campus extraction
Country suffix append
YAML: strip_patterns
Online/Virtual strip
Slash-separated cities

intake.py

Month extraction
Session->month map
Autumn->Mar, Spring->Jul
Summer->Nov (AU unis)

cricos_code.py

CRICOS code regex
PDF pipeline integration
AU universities only
scraped_field_evidence

Others

degree_level.py
description.py
eligibility.py
gemini_primary.py (AI)
ai_fallback.py

LAYER 6 -- Per-University YAML Config System

scraper_config/ (backend-py/scrapper_config/)

defaults.yaml -- conservative global defaults (change needs full regression sweep)
unis/<slug>.yaml -- per-university overrides (30+ stubs: uwa, acap, bond, csu...)
config/schema.py -- Pydantic UniConfig: DiscoveryConfig + ExtractionConfig
config/loader.py -- deep-merge: defaults -> DB scrape_config -> per-uni YAML
config/context.py -- ContextVar[UniConfig] scoped per scrape job
Key YAML fields: strip_title_suffixes, reject_sentence_patterns (duration),
strip_patterns (location), domestic_only, fallback_subdomains,
always_sitemap_supplement, pte_blocklist, domestic_text_filter
Architecture rule: ALL per-university quirks go in YAML -- NEVER in global code

LAYER 7 -- Database (PostgreSQL)

Core Tables

universities -- id, name, website, country,
scrape_config (JSONB)
courses -- live courses: name,
degree_level, fee, duration,
location, mode, description
intakes -- intake months per course
fees -- fee breakdown per course
english_requirements -- IELTS/PTE/TOEFL bands
academic_requirements -- GPA, score, level
scholarships -- scholarship info per course

Scraping & Operations Tables

scraped_courses -- staging: pending->approved
scraped_field_evidence -- per-field provenance
method, source, confidence
scrape_runtime_jobs -- job stats: found/staged/cost
scrape_run_alerts -- rule-based alerts per run
discovery_failure_alerts -- Tier-7 fired alerts
gemini_call_log -- every AI call logged
import_jobs -- Excel import history
change_history -- field-level before/after log
SQL views: v_gemini_cost_by_university,
v_cricos_coverage_au, and 3 more

DEPLOYMENT -- Production (DigitalOcean)

Server: 159.65.152.72 | Ubuntu 24.04 | Git: /root/University-and-Course-data

GitHub: <https://github.com/Studyinfocentre/University-and-course-managment>
Deploy: git pull origin main + systemctl restart uni-api-py.service uni-celery.service
FastAPI: uni-api-py.service (Uvicorn port 8000) -- Nginx proxies /api -> 127.0.0.1:8000
Celery: uni-celery.service (4 workers + beat scheduler)
DB access: sudo -u postgres psql -d university_portal

MONOREPO STRUCTURE (pnpm workspaces)

Directory Layout

artifacts/university-portal/	React+Vite UI
src/pages/	Page components
src/components/	Shared UI
src/context/auth.tsx	Auth context
backend-py/	Python backend
app/main.py	FastAPI entry
app/routers/	API handlers
app/services/scrapper/	Scraper engine
extractors/	15+ extractors
config/	YAML config pkg
pipelines/	PDF pipeline
scraper_config/unis/	Per-university YAMLs
scripts/	Ops scripts
tests/	Pytest suite

Key Scripts (backend-py/scripts/)

bulk_approve.py	Batch promote staged
capture_baseline.py	Snapshot all unis
regression_sweep.py	Diff snapshots
bond_enrich.py	Bond JSON API enrich
csu_promotion_diagnostic.py	
week4_preflight.sh	4-gate pre-flight
week5_preflight.sh	Week-5 variant
week4_cost_projection.sql	
apply_migration_*.py	Schema migrations
cricos_coverage_diagnostic.py	
check_gemini_model.py	Verify AI model
bulk_upload	for Excel imports